

Week 1: Introduction

CSCI 447: Machine Learning– Fall 2026

1

Dr. John Sheppard

- Distinguished Professor of Computer Science, MSU (2008)
- Research or Adjunct faculty at Johns Hopkins since 1994
- Director, Numerical Intelligent Systems Laboratory
- BS in Computer Science (Southern Methodist University)
- MS, PhD in Computer Science (Johns Hopkins University)
- 20 years in industry – ARINC Inc. (Defense Aerospace)
- 20 years in full-time academia – JHU and MSU
- Interests: Probabilistic models, distributed optimization, ethical and explainable AI
- Founding Member: IEEE Computational Intelligence Society's Technical Committee on Ethical, Legal, Social, Environment, and Human Dimensions of AI/CI (SHIELD)
- IEEE Life Fellow



2

Administrative

- Professor: Dr. John W. Sheppard
 - Class: 1:10 – 2:00 pm, MWF, GH 100
 - Office: GH 334C
 - Office Hours: 2:00 – 3:00 pm, MWF
 - TA TBD
- Course materials are in Canvas
 - Syllabus
 - Assignments
 - Grades
 - Lecture Slides (after being presented)
- Attendance is expected but will not be checked. Note that exam questions may come from material only discussed in class.

Grading

- Four Programming Assignments (15% each)
 - Will be done in teams of 2 or 3
 - Any language permitted, as long as all on the team are proficient
 - Algorithm libraries not permitted
 - AI-based coding not permitted
 - No code will be turned in
 - Only deliverable is a report in JMLR format
- Two Exams
 - Midterm (15%) and Final (25%)
 - No electronic resources permitted, including cell phones, laptops, tablets
 - No cheat sheets permitted
 - Objective, Project-related, Short-answer

Policy on GenAI

- This is not a course on how to use AI. It is a course on understanding, implementing, and analyzing ML algorithms.
- Using GenAI on the assignments is strictly prohibited
 - No code generation (all coding needs to be yours)
 - No prose generation (I want to know what you think, not what AI thinks)
 - No prose summarization or clean-up (these are easy to abuse)
- You may use basic spell checkers and grammar checkers on reports
- TurnItIn will be turned on
 - Checks for plagiarism
 - Checks for AI usage
- The use of GenAI will constitute academic misconduct and will be reported to the Dean of Students

What is Machine Learning?

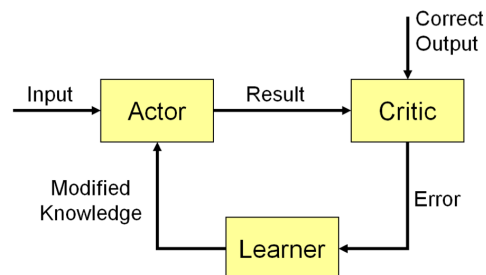
- A computer program that improves performance over time.
- Mapping of data into a model (Data → Model).
- Learning behavior from data.
- Discovering solutions to hard problems.

Definition (Mitchell): A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P if its performance on tasks T as measured by P improves with E .

Definition (Alpaydin): Machine learning is programming computers to optimize a performance criterion using example data or past experience.

Types of Machine Learning

Supervised Learning

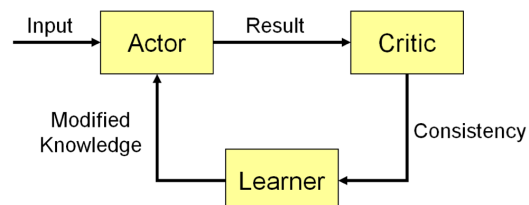


[This is not to be confused with the so-called “actor-critic” model in reinforcement learning.]

- Online learning
- Offline learning
- Active learning

Types of Machine Learning

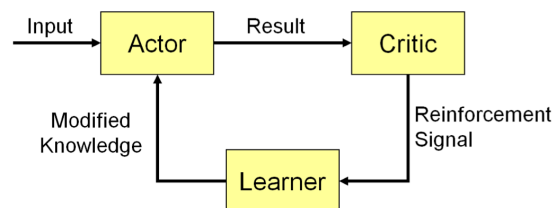
Unsupervised Learning



- Clustering
- Dimensionality reduction
- Association analysis
- Representation learning

Types of Machine Learning

Reinforcement Learning

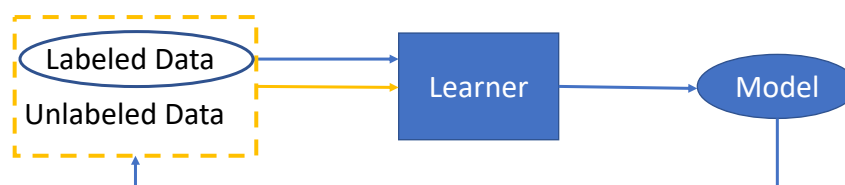


[This topic is covered in CSCI 446 – Artificial Intelligence, and will only be touched on lightly in this class.]

- Sequential decision problems
- Requires a reinforcement signal rather than ground truth

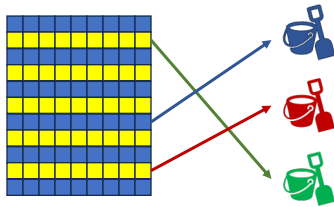
Semi-Supervised Learning

- An approach that mixes labeled and unlabeled data.
- One common approach is called “self training.”

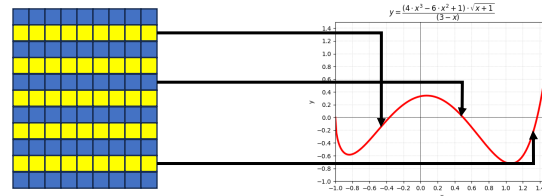


Machine Learning Problem Types

Classification



Regression



11

Example: The Winnow-2 Algorithm

Training (supervised)

1. Receive a data instance (numeric)
2. Make a prediction (classification)
3. Is the prediction correct? If not,
 - $0 \Rightarrow 1$: Promotion
 - $1 \Rightarrow 0$: Demotion

Classification

- Model:

$$f(\mathbf{x}) = \sum_{i=1}^d w_i x_i$$

- For threshold θ ,

$$h(\mathbf{x}) = \begin{cases} 1 & f(\mathbf{x}) > \theta \\ 0 & \text{otherwise} \end{cases}$$

12

Example: The Winnow-2 Algorithm

Classification

- Model:

$$f(\mathbf{x}) = \sum_{i=1}^d w_i x_i$$

- For threshold θ ,
(hyperparameter)

$$h(\mathbf{x}) = \begin{cases} 1 & f(\mathbf{x}) > \theta \\ 0 & \text{otherwise} \end{cases}$$

Training (supervised)

1. Receive a data instance $\mathbf{x} \in \mathcal{D}$
(numeric)
2. Make a prediction
(classification)
3. Is the prediction correct? If not,
 - $0 \xrightarrow{\text{increase}} 1$: Promotion
 - $1 \xrightarrow{\text{decrease}} 0$: Demotion

Example: The Winnow-2 Algorithm

Promotion

If classify as 0 when correct class is 1, promote:

$$w_i = \begin{cases} \alpha w_i & \text{if } x_i = 1 \\ w_i & \text{if } x_i = 0 \end{cases}$$

Demotion

If classify as 1 when correct class is 0, demote

$$w_i = \begin{cases} w_i / \alpha & \text{if } x_i = 1 \\ w_i & \text{if } x_i = 0 \end{cases}$$

Note that α is a hyperparameter.

Worked Example

$$f(\mathbf{x}) = (x_1 \vee x_2) \wedge x_3$$

$$\mathbf{w} = [1.0 \ 1.0 \ 1.0]$$

$$\alpha = 2$$

$$\theta = 0.5$$

$$f(\mathbf{x}) = \sum_{i=1}^d w_i x_i$$

$$h(\mathbf{x}) = \begin{cases} 1 & f(\mathbf{x}) > \theta \\ 0 & \text{otherwise} \end{cases}$$

$\mathcal{D} =$

x_1	x_2	x_3	$f(\mathbf{x})$	
0	0	0	0	→ Ok
0	0	1	0	[1.0 1.0 0.5]
0	1	0	0	[1.0 0.5 0.5]
0	1	1	1	→ Ok
1	0	0	0	[0.5 0.5 0.5]
1	0	1	1	⋈
1	1	0	0	
1	1	1	1	[0.25 0.25 0.5]



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

15

Why Does Machine Learning Work?

Definition: Consider any concept learning algorithm L for a set of instances X . The **inductive bias** of L is any minimal set of assertions B such that for any target concept c and corresponding training examples D_c ,

$$\forall x_i \in X, (B \wedge D_c \wedge x_i) \vdash L(x_i, D_c).$$

Theorem: A learner that makes no *a priori* assumptions regarding the identity of the target concept has no rational basis for classifying any unseen instances.

Interpretation:

An unbiased learner cannot learn!



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

16

Types of Inductive Bias

- **Representation Bias** – How the type of model defines the hypothesis (i.e., search) space for the learner.
- **Preference Bias** – How the learner searches the hypothesis space and decides which hypotheses are better than others.

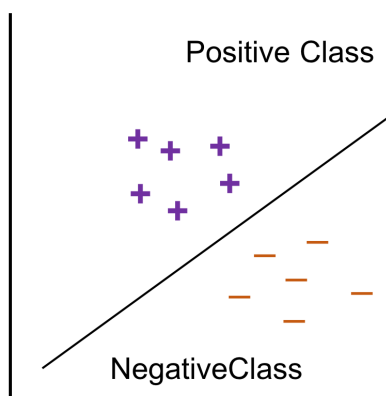
Example: Winnow-2

Winnow-2 has a *linear* representation bias

$$f(\mathbf{x}) = \sum_{i=1}^d w_i x_i > \theta$$

17

Linear Representation Bias



18

Linear Representation Bias

Let weights be $[0.5 \ 0.5]$

Assume a threshold of $\theta = 0.5$

x_1	x_2	$f(\mathbf{x})$	$f(\mathbf{x}) = (x_1 \wedge x_2)$
0	0	0	
0	1	0	
1	0	0	
1	1	1	

19

Linear Representation Bias

Exclusive OR has no linear separator.

Concept does not match the linear representation bias.

x_1	x_2	$f(\mathbf{x})$	$f(\mathbf{x}) = (x_1 \oplus x_2)$
0	0	0	
0	1	1	
1	0	1	
1	1	0	

20

Preference Bias

- Also sometimes referred to as “selection bias,” but this term can be confused with the sampling bias of the same name.
- Emphasizes that machine learning is a *search* process.
- Also emphasizes the search in machine learning involves *optimization*.
- The objective is to maximize performance

Examples

- Maximizing a performance measure / Minimizing a loss function.
- Making the model as simple as possible (Principle of Parsimony, aka Occam’s Razor): this avoids “overfitting.”

No Free Lunch

- Consider machine learning as an optimization problem.
- Define $d_c = \{\langle d_c^x(1), d_c^y(1) \rangle, \dots, \langle d_c^x(m), d_c^y(m) \rangle\}$.
- Thus the i^{th} mapping is $\langle d_c^x(i), d_c^y(i) \rangle$.
- We wish to compare two algorithms: $a1$ and $a2$.
- We compare relative to learning a target function f .
- We want to consider $P(d_c^y|f, m, a)$.

Theorem (Wolpert & Macready): For any pair of algorithms $a1$ and $a2$,

$$\sum_f P(d_c^y|f, m, a1) = \sum_f P(d_c^y|f, m, a2)$$

Inductive Learning Hypothesis

- “Any **hypothesis** found to approximate the **target function** well over a sufficiently large **set of training examples** will also approximate the **target function** well over other **unobserved examples**” (Mitchell, 1997)
- The end goal is to learn a model capable of **generalizing** on examples not used in the training process

Evaluating Hypotheses

- The **true error** of hypothesis h with respect to target function f and distribution $\mathcal{D}$ is the probability that H will misclassify an instance drawn at random according to $\mathcal{D}$

$$error_{\mathcal{D}}(h) = P_{x \in \mathcal{D}}[f(x) \neq h(x)]$$

- The **sample error** of h with respect to f and data sample $\mathcal{S}$ is the proportion of examples h misclassifies

$$error_{\mathcal{S}}(h) = \frac{1}{n} \sum_{x \in \mathcal{S}} \delta(f(x), h(x)), \text{ where } \delta(f(x), h(x)) = \begin{cases} 1 & f(x) \neq h(x) \\ 0 & f(x) = h(x) \end{cases}$$

Risk and Loss Functions

Risk function: $\ell(\Theta) = \sum_i L(g_\Theta(x^{(i)}), r^{(i)}) + \lambda \rho(\Theta)$

- 0/1-Loss: $L_{01} = \begin{cases} 1 & g_\Theta(x) \neq r \\ 0 & g_\Theta(x) = r \end{cases}$
- Square loss: $L_2(g_\Theta(x), r) = (g_\Theta(x) - r)^2$
- Hinge loss: $L_{\text{hinge}}(g_\Theta(x), r) = \max\{0, 1 - g_\Theta(x) \cdot r\}$
- Log Loss: $L_{\log}(g_\Theta(\mathbf{x}), r) = y \log(g(\mathbf{x})) + (1 - r) \log(1 - g(\mathbf{x}))$,
where $g(\mathbf{x}) = \frac{1}{1 + e^{-g_\Theta(\mathbf{x})}}$



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

25

Bias-Variance Tradeoff

- Let $\mathcal{X} = \{\mathbf{x}^t, r^t\}_{t=1}^n \sim p(\mathbf{x}, r)$, where p is unknown.
- Learn a model $g(\mathbf{x})$.
- Assume we measure loss via expected squared error over p

$$E[\Theta|\mathcal{X}] = \frac{1}{n} \sum_{t=1}^n [r^t - g(\mathbf{x}^t|\Theta)]^2.$$

- Then this expected squared error can be rewritten as

$$E[(r - g(\mathbf{x}))^2 | \mathbf{x}] = \underbrace{E[(r - E[r|\mathbf{x}])^2 | \mathbf{x}]}_{\text{noise}} + \underbrace{(E[r|\mathbf{x}] - g(\mathbf{x}))^2}_{\text{squared error}}$$

which can be rewritten through re-arrangement as (p. 72, Eq. 11)

$$E_x[(E[r|\mathbf{x}] - g(\mathbf{x}))^2 | \mathbf{x}] = \underbrace{(E[r|\mathbf{x}] - E_x[g(\mathbf{x})])^2}_{\text{bias}} + \underbrace{E_x[(g(\mathbf{x}) - E_x[g(\mathbf{x})])^2]}_{\text{variance}}$$



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

26

Take Aways

- Don't look for a panacea of learning algorithms.
- Understand your problem and your data.
- Pick the model representation to balance bias and variance
- Consider the effect of the performance (i.e., loss) function on the behavior of the learning algorithm.
- Pick an algorithm that is well-suited to optimize that performance (i.e., loss) function.